

Predicción de fallos APS en camiones Scania

Mantenimiento predictivo con datos tabulares, coste asimétrico, explicabilidad y robustez

Mauro Valls Vidal
Alejandro Parra Sánchez
Jordi Blasco Lozano
Alejandro Martínez Riquelme

Grado en Ingeniería en Inteligencia Artificial
Escuela Politécnica Superior, Universidad de Alicante
Aprendizaje Avanzado

Curso 2025–2026

Resumen

Este trabajo presenta un estudio de aprendizaje automático para detectar fallos relacionados con el sistema APS de camiones Scania. La tarea se formula como clasificación binaria sobre datos tabulares industriales con 170 variables predictoras, valores faltantes, variables anonimizadas y una clase positiva muy minoritaria. El objetivo es identificar fallos APS y reducir el riesgo de falsos negativos. Los modelos se comparan mediante `Pipeline`, `GridSearchCV` y validación cruzada estratificada de 5 folds usando únicamente train. El mejor estimador se reentrena con todo train mediante `refit` y test queda reservado para la evaluación final. Se comparan un baseline, modelos lineales, SVM lineal, árboles y ensembles. El modelo principal seleccionado es `HistGradientBoosting`, con PR-AUC media de 0.881 en validación cruzada. En test obtiene PR-AUC de 0.922, F1 positivo de 0.842, recall positivo de 0.752, 13 falsos positivos y 93 falsos negativos. La memoria añade análisis de coste asimétrico, anomalías, importancia por permutación, robustez, errores y shift. La conclusión es que `HistGradientBoosting` es el modelo principal defendible, mientras que `logistic_class_weight` queda como alternativa operativa si se prioriza coste.

Palabras clave Mantenimiento predictivo, Scania APS, clasificación binaria, datos tabulares, desbalanceo, PR-AUC, coste asimétrico, explicabilidad, robustez.

Índice

1. Introducción	3
2. Contexto y trabajo relacionado	3
2.1. Mantenimiento predictivo	3
2.2. Clasificación tabular y desbalanceo	3
2.3. Coste, explicabilidad y robustez	4
3. Descripción del problema y los datos	4
3.1. Definición del problema	4
3.2. Origen y características del dataset	4
3.3. Análisis exploratorio	5
3.3.1. Distribución de clases y accuracy engañosa	5
3.3.2. Valores faltantes	6
3.3.3. Escalas, outliers y correlaciones	7
3.4. Problemáticas identificadas	9
3.5. Flujo reproducible de notebooks	10

4. Metodología	11
4.1. Conceptos técnicos usados	11
4.2. Protocolo experimental	11
4.3. Modelos comparados	12
4.4. Métricas de evaluación	12
4.5. Reproducibilidad	12
5. Experimentos y resultados	13
5.1. Modelado supervisado	13
5.1.1. Resultados de validación cruzada	13
5.1.2. Selección del modelo principal	13
5.2. Resultados finales en test	13
5.3. Desbalanceo y coste	14
5.4. Análisis de anomalías	15
5.5. Explicabilidad, robustez y errores	16
5.5.1. Importancia por permutación	16
5.5.2. Análisis de errores	17
5.5.3. Robustez ante ruido	18
5.5.4. Shift entre train y test	18
6. Discusión	19
6.1. Por qué HistGradientBoosting es el modelo principal	19
6.2. PR-AUC frente a coste	19
6.3. Explicabilidad con variables anonimizadas	20
6.4. Limitaciones	20
6.5. Trabajo futuro	20
7. Conclusiones	20
8. Referencias	21
A. Anexo A. Archivos principales del flujo	22
B. Anexo B. Reproducción	22

1. Introducción

El mantenimiento predictivo busca anticipar fallos antes de que se conviertan en averías costosas. En vehículos industriales, una detección temprana puede reducir paradas, reparaciones innecesarias y riesgos operativos. Este proyecto se centra en el sistema APS de camiones Scania, relacionado con el suministro de aire a presión. Un fallo de este sistema puede afectar al funcionamiento del vehículo y generar costes importantes.

El problema tiene una dificultad práctica clara. Los fallos APS reales son poco frecuentes, por lo que la clase positiva aparece muy poco frente a la clase negativa. En este contexto, un modelo puede obtener una accuracy alta aunque ignore casi todos los fallos reales. Por esta razón, el proyecto no se evalúa solo con accuracy. Se usan métricas centradas en la clase positiva como recall, F1 y PR-AUC, junto con un coste asimétrico que penaliza más los falsos negativos.

La motivación del aprendizaje automático es aprovechar señales presentes en sensores, contadores y agregados internos del camión. Aunque las variables están anonimizadas, sus patrones estadísticos pueden ayudar a distinguir casos APS de casos no APS. La anonimización limita la interpretación física directa, pero no impide construir un modelo predictivo si el protocolo experimental se diseña con cuidado.

El objetivo del trabajo es construir y evaluar un flujo reproducible de aprendizaje automático para detectar fallos APS. El sistema no se presenta como una solución de producción. Es un estudio académico y técnico que compara modelos, analiza errores y discute limitaciones.

Las contribuciones del trabajo se resumen en cuatro puntos. Primero, se construye un flujo reproducible de preparación, exploración, modelado y auditoría para el dataset APS. Segundo, se comparan modelos de familias distintas bajo un protocolo común con `Pipeline`, `GridSearchCV` y validación cruzada de 5 folds. Tercero, se incorpora una lectura operativa del problema mediante coste asimétrico, análisis de falsos negativos y comparación de estrategias sensibles al desbalanceo. Cuarto, se añaden análisis complementarios de anomalías, explicabilidad, robustez y shift que permiten interpretar el comportamiento del modelo más allá de una única métrica final.

2. Contexto y trabajo relacionado

2.1. Mantenimiento predictivo

El mantenimiento predictivo consiste en usar datos históricos del sistema para estimar si puede aparecer un fallo antes de que ocurra una avería grave. La idea no es esperar a que el camión falle, como ocurre en el mantenimiento correctivo, ni revisar siempre cada cierto tiempo aunque no haya indicios de problema, como ocurre en el mantenimiento preventivo tradicional.

En este enfoque, las revisiones se intentan adaptar al estado real del vehículo. Si los datos de los sensores indican un comportamiento anómalo, el sistema puede recomendar una revisión antes de que el fallo cause una parada o una reparación más costosa.

En camiones y otros vehículos industriales, esto es especialmente útil porque una avería puede dejar el vehículo inmovilizado, generar retrasos y aumentar los costes de mantenimiento. Sin embargo, el modelo debe evaluarse con cuidado. Un falso positivo implica revisar un camión que realmente no tenía fallo APS. Un falso negativo implica no detectar un fallo real. En este proyecto, el falso negativo es el error más grave porque puede dejar pasar una avería que debería haberse anticipado.

2.2. Clasificación tabular y desbalanceo

El dataset APS es tabular. Cada fila representa un caso de mantenimiento y cada columna describe una señal numérica. Este tipo de problema suele beneficiarse de árboles, bosques aleatorios y métodos de boosting, aunque los modelos lineales siguen siendo útiles como referencia.

El desbalanceo es una característica central del problema. Como hay muy pocos casos positivos, un modelo podría acertar muchos negativos y aun así fallar justo en los fallos APS que interesa

detectar. Por eso la accuracy puede dar una impresión demasiado optimista. En su lugar se usan métricas centradas en la clase positiva, como recall, F1 y PR-AUC. PR-AUC es especialmente útil porque evalúa cómo de bien el modelo prioriza los casos positivos cuando estos son escasos.

2.3. Coste, explicabilidad y robustez

En un problema industrial no todos los errores pesan igual. El dataset define un coste de 10 para falsos positivos y de 500 para falsos negativos. Este esquema no captura toda la complejidad real del mantenimiento, pero permite estudiar el impacto operativo de los errores.

La explicabilidad también importa. En este proyecto se usa importancia por permutación, que consiste en desordenar una variable y medir cuánto empeora el modelo, para estimar qué variables influyen más en sus predicciones. Esta técnica no demuestra causalidad. Sirve para auditar dependencias del modelo y detectar si el rendimiento depende de pocas variables.

La robustez se analiza añadiendo ruido a variables numéricas. Esta prueba no equivale a una validación de producción, pero ayuda a observar si pequeñas perturbaciones degradan mucho el rendimiento. Gracias a este análisis se puede valorar si el modelo sería sensible a errores, ruido o pequeñas variaciones en las mediciones de los sensores.

3. Descripción del problema y los datos

3.1. Definición del problema

El sistema propuesto está pensado como una herramienta de apoyo para equipos de mantenimiento, responsables de flota o personal técnico que necesite priorizar revisiones de camiones. La salida del modelo no debe interpretarse como una decisión automática de reparación. Su función sería señalar casos con riesgo de fallo APS para que una persona experta pueda revisarlos con más información.

La tarea se define como clasificación binaria. Cada observación x_i contiene 170 variables numéricas procedentes del sistema del camión. La etiqueta y_i toma valor 0 si el caso no está relacionado con APS y valor 1 si el fallo sí está relacionado con APS.

$$f(x_i) \rightarrow \hat{y}_i \in \{0, 1\}$$

Las consecuencias de los errores no son simétricas. Un falso positivo implica revisar un camión que finalmente no presenta fallo APS. Esto puede aumentar el coste de inspección y generar trabajo adicional. Un falso negativo implica no detectar un fallo APS real. Este error es más delicado porque puede retrasar una intervención necesaria y aumentar el coste operativo posterior. Por ello, el proyecto presta especial atención al recall de la clase positiva, al coste y al análisis de falsos negativos.

3.2. Origen y características del dataset

El dataset usado es *APS Failure at Scania Trucks* del UCI Machine Learning Repository. Es un dataset tabular suficientemente rico para el objetivo del proyecto, ya que contiene decenas de miles de instancias y 170 variables predictoras. Proporciona una partición oficial de entrenamiento y otra de test. La Tabla 1 resume sus características principales.

Tabla 1: Características principales del dataset

Elemento	Valor
Train	60000 filas
Test	16000 filas
Variables predictoras	170
Tipo de datos	Tabulares numéricos
Clase negativa original	neg
Clase positiva original	pos
Codificación usada	neg como 0 y pos como 1
Coste de falso positivo	10
Coste de falso negativo	500

Las variables predictoras están anonimizadas. Proceden de sensores, contadores o agregados internos del camión, pero no se conoce su significado físico exacto. Esta anonimización es una limitación importante, porque dificulta interpretar por qué una variable concreta está asociada al fallo. Por ello, la memoria distingue entre importancia estadística e interpretación causal.

Los valores faltantes aparecen marcados como **na** en los archivos originales. Esos valores no son valores faltantes reales al cargarlos directamente, sino texto que indica ausencia de dato. Por eso, durante la preparación se convierten a valores faltantes reconocibles por las librerías de análisis, de forma que puedan tratarse correctamente mediante imputación dentro de los **Pipeline**.

La variable objetivo se transforma a formato binario. La clase **neg** se codifica como 0 y representa casos no relacionados con APS. La clase **pos** se codifica como 1 y representa fallos relacionados con APS. Esta clase positiva es muy minoritaria. En train, representa alrededor del 1.67 % de los casos.

3.3. Análisis exploratorio

3.3.1. Distribución de clases y accuracy engañosa

El primer resultado del EDA es el fuerte desbalanceo. La mayoría de observaciones pertenecen a la clase negativa. Un clasificador que predijera siempre la clase mayoritaria alcanzaría una accuracy muy alta, pero no detectaría fallos APS reales. Esta observación justifica que accuracy quede como métrica secundaria.

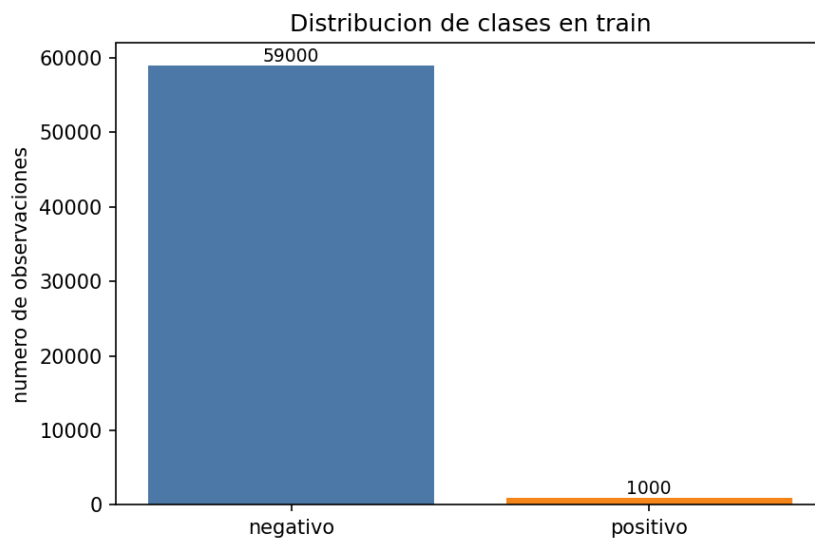


Figura 1: Distribución de clases en train

La Figura 1 muestra la desproporción entre negativos y positivos. Esta diferencia explica por

qué el baseline puede parecer razonable si solo se mira accuracy, aunque sea incapaz de detectar la mayoría de fallos APS.

La evaluación se centra en recall positivo, F1 positivo, PR-AUC, balanced accuracy y coste. Estas métricas son más adecuadas que accuracy porque el dataset está muy desbalanceado y la clase importante es la positiva.

El recall positivo indica qué parte de los fallos APS reales consigue detectar el modelo. Es importante porque un recall bajo significa que muchos fallos quedan sin identificar. El F1 positivo combina precisión y recall en una sola medida, por lo que ayuda a valorar si el modelo detecta fallos sin generar demasiadas falsas alarmas. PR-AUC mide la capacidad del modelo para priorizar los casos positivos frente a los negativos, algo especialmente útil cuando hay pocos positivos. Balanced accuracy calcula el acierto teniendo en cuenta ambas clases de forma más equilibrada. Por último, el coste traduce los errores a impacto operativo, penalizando mucho más los falsos negativos que los falsos positivos.

3.3.2. Valores faltantes

El dataset contiene muchos valores faltantes. Algunas variables presentan proporciones muy elevadas. En train, variables como `br_000`, `bq_000`, `bp_000`, `bo_000`, `ab_000` y `cr_000` superan el 77% de valores faltantes en varios casos. Esto significa que, en esas columnas, más de 77 de cada 100 filas no tienen dato registrado.

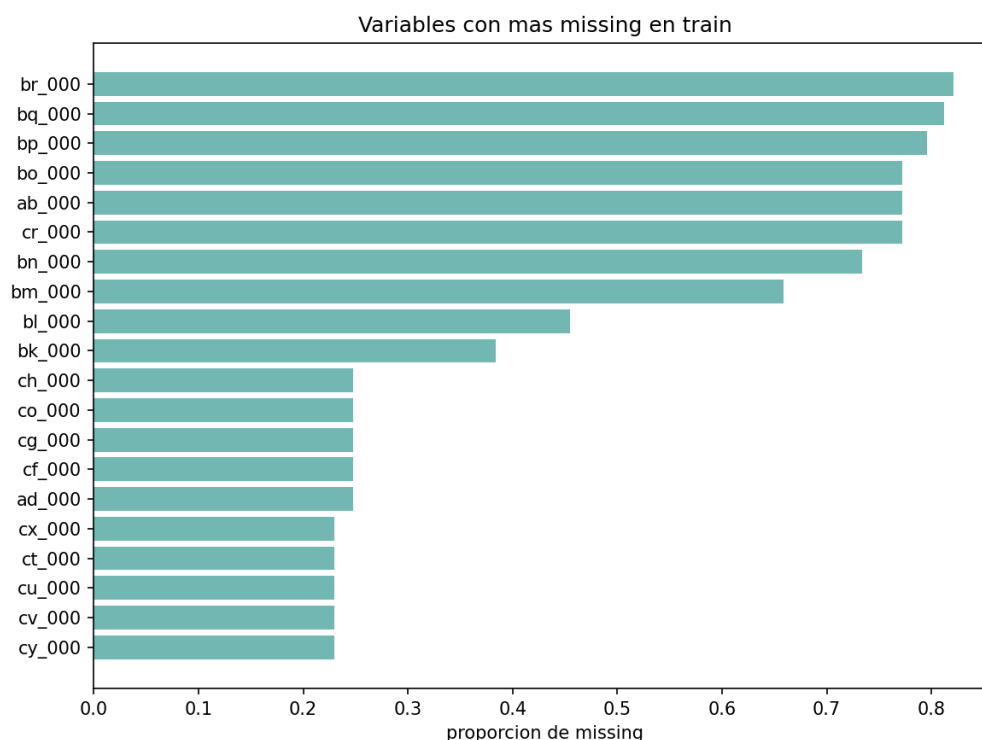


Figura 2: Variables con mayor proporción de valores faltantes en train

La Figura 2 muestra el porcentaje total de valores faltantes por variable en todo train. El eje horizontal representa una proporción entre 0 y 1. Por ejemplo, un valor de 0.8 significa que aproximadamente el 80% de los registros de esa variable están vacíos. Esta figura permite ver qué columnas tienen más ausencia de datos en general.

Además del missing total, el EDA analiza si la ausencia de datos cambia según la clase. Esto es importante porque un valor faltante no siempre significa simplemente un error de registro. En datos de sensores, una variable puede faltar porque un sensor no se activó, porque una condición del sistema no se produjo o porque el proceso de captura no registró esa señal. Por eso, la ausencia

de datos también puede aportar información indirecta sobre el estado del camión.

En algunas variables, como `ac_000`, `ad_000`, `ae_000`, `af_000` y `ak_000`, los valores faltantes son más frecuentes en la clase positiva. En otras, como `br_000`, `bq_000`, `bp_000`, `bo_000` y `bn_000`, la ausencia aparece mucho más en la clase negativa.

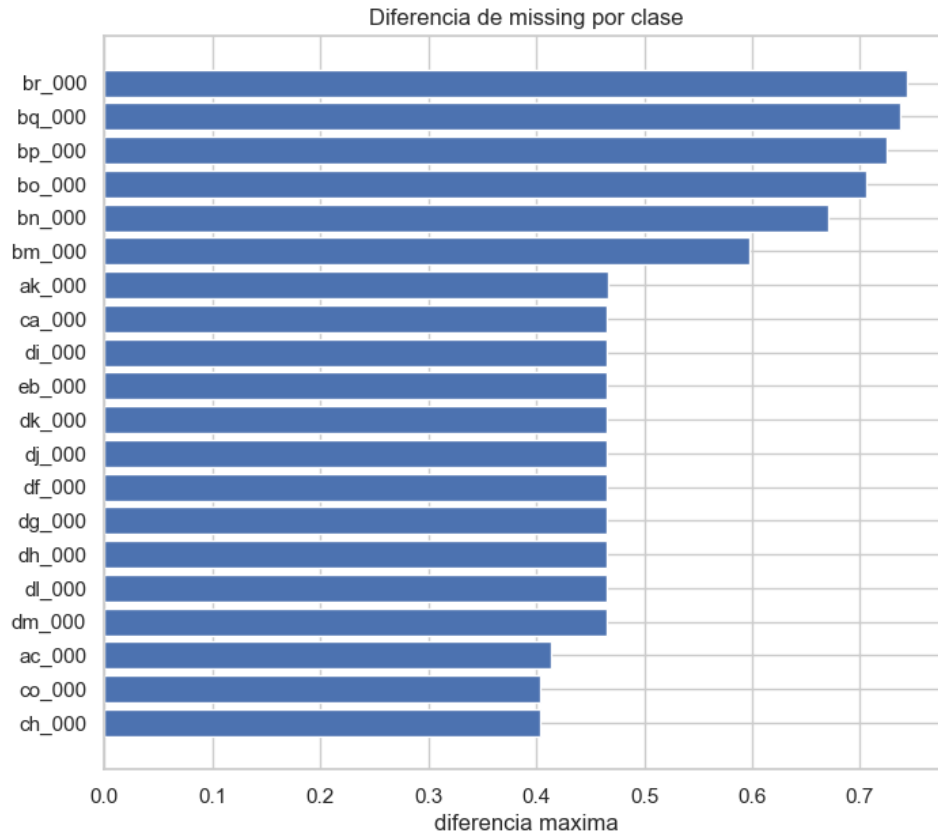


Figura 3: Variables con mayor diferencia de valores faltantes entre clases

La Figura 3 no muestra el missing total, sino la diferencia máxima de missing entre clases. Su eje horizontal también es una proporción entre 0 y 1. Por ejemplo, una diferencia de 0.7 significa que la proporción de missing entre negativos y positivos cambia en unos 70 puntos porcentuales. Esta figura permite ver qué variables están vacías de forma distinta según la clase.

La lectura conjunta de ambas figuras es importante. La primera muestra qué columnas están más incompletas. La segunda muestra si esa ausencia está repartida de forma desigual entre negativos y positivos. Por tanto, los missing values no son solo un problema de limpieza, sino una característica relevante de la estructura del dataset.

Por este motivo, no se eliminan columnas de forma manual desde el EDA. Esa decisión podría descartar información útil o introducir criterios difíciles de justificar. En su lugar, los valores faltantes se tratan mediante imputación con mediana dentro de cada `Pipeline`. Así, durante la validación cruzada, cada fold calcula la mediana solo con su parte de entrenamiento y se evita fuga de información.

3.3.3. Escalas, outliers y correlaciones

Las variables tienen escalas muy diferentes. Algunas se concentran cerca de cero y otras alcanzan valores muy altos. También aparecen valores extremos. Según el análisis IQR, variables como `az_007`, `dr_000`, `dq_000`, `bn_000`, `ba_008`, `cj_000` y `aj_000` tienen tasas elevadas de outliers.

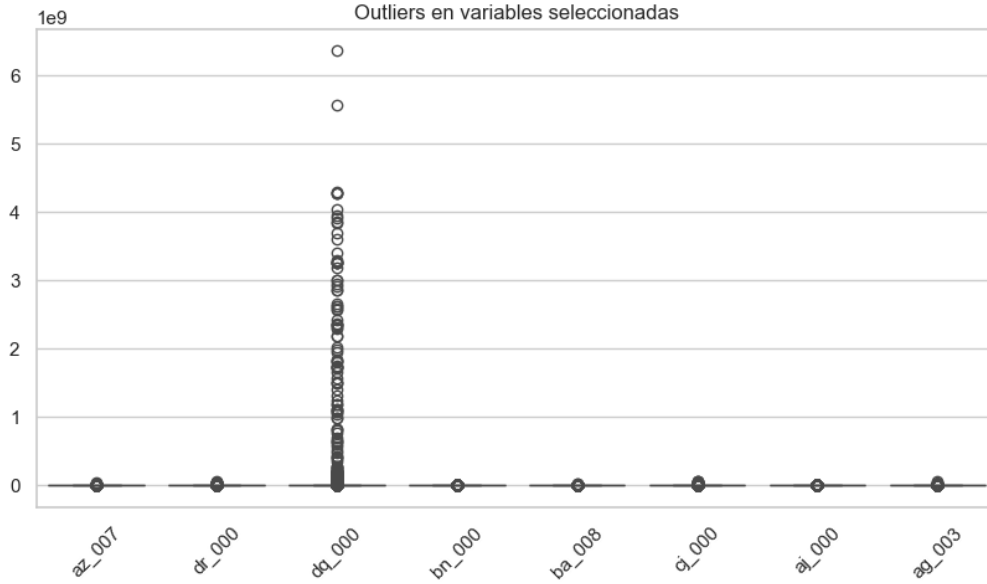


Figura 4: Boxplots de variables seleccionadas para revisar escalas y valores extremos

La Figura 4 ilustra la diferencia de escalas y la presencia de valores extremos en algunas variables. La visualización no es completamente cómoda porque `dq_000` alcanza valores mucho más altos que el resto y comprime visualmente las demás distribuciones. Aun así, esta evidencia justifica el escalado en modelos sensibles a la magnitud de las variables y la comparación con modelos basados en árboles.

Para complementar esa visualización, la Tabla 2 resume algunas variables con mayor proporción de outliers según la regla IQR. Por este motivo, se añade la Tabla 2, que resume algunas de las variables con mayor proporción de outliers según la regla IQR.

Tabla 2: Ejemplos de variables con mayor proporción de outliers según la regla IQR

Variable	Outliers	Tasa	Q1	Q3
az_007	14235	0.240	0	0
dr_000	13007	0.227	0	0
dq_000	12870	0.225	0	0
bn_000	3509	0.219	171720	493100
ba_008	12580	0.212	0	1820
cj_000	12540	0.210	0	0
aj_000	12452	0.210	0	0
ag_003	12435	0.210	0	0
dy_000	11867	0.207	0	36
dx_000	11572	0.202	0	8764

La tabla muestra que los valores extremos no se concentran en una única variable. Varias columnas tienen más de un 20% de registros considerados outliers por la regla IQR. En muchas de ellas, como `az_007`, `dr_000`, `dq_000`, `cj_000`, `aj_000` y `ag_003`, tanto Q1 como Q3 son 0. Esto significa que la mayoría de registros están concentrados en cero, pero existe un grupo relevante de casos con valores distintos y alejados de ese comportamiento habitual.

Este patrón es razonable en datos industriales. Algunas señales pueden permanecer inactivas durante funcionamiento normal y activarse solo en condiciones concretas. Por eso no se eliminan estos valores automáticamente. Pueden representar información útil sobre estados anómalos del camión y ayudan a justificar el uso de modelos capaces de manejar distribuciones asimétricas y con colas largas.

Las correlaciones muestran redundancia entre algunas variables. Por ejemplo, `aa_000` y `bt_000` tienen una correlación prácticamente perfecta en la muestra analizada. También aparecen relaciones altas entre `ci_000`, `ao_000`, `bj_000`, `bi_000` y otras variables. Esta redundancia es habitual en datos de sensores.

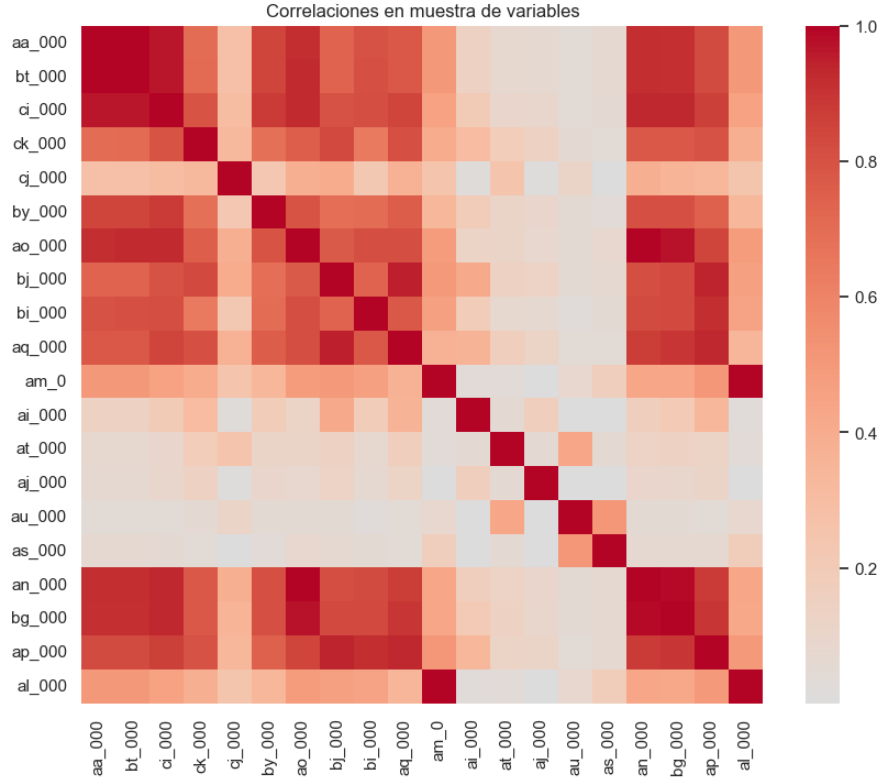


Figura 5: Matriz de correlaciones en una selección de variables numéricas

La Figura 5 confirma que algunas señales están muy relacionadas. Esta redundancia no se elimina manualmente, porque puede ser propia de datos de sensores y cada familia de modelos puede gestionarla de forma distinta.

Estas observaciones motivan el uso de escalado en modelos sensibles a escala, como regresión logística y SVM lineal. Los modelos basados en árboles no necesitan el mismo tratamiento, aunque todos mantienen imputación dentro del pipeline.

3.4. Problemáticas identificadas

En este proyecto se identifican varias problemáticas reales asociadas al uso de sistemas de aprendizaje automático en mantenimiento predictivo.

El primer problema es el desbalanceo de clase. La clase positiva es minoritaria y representa aproximadamente el 1.67 % de train. Esto implica que un modelo puede acertar muchos negativos y aun así fallar en el objetivo real del sistema, que es detectar fallos APS. Por este motivo se usan PR-AUC, recall positivo, F1 positivo, balanced accuracy y coste, en lugar de depender solo de accuracy.

El segundo problema son los datos escasos en la clase positiva. Aunque el dataset tiene 60000 filas de train, los casos APS son pocos en términos relativos. Esto dificulta aprender la frontera de decisión de la clase de interés y hace que cada falso negativo tenga importancia. La comparación con el baseline permite comprobar si los modelos aprenden señal útil o si solo reflejan la clase mayoritaria.

El tercer problema es el coste asimétrico. En el dominio de mantenimiento, revisar un camión sin fallo no tiene el mismo impacto que dejar pasar una avería real. El dataset formaliza esta

diferencia con coste 10 para falsos positivos y 500 para falsos negativos. Por eso se estudia una estrategia alternativa, `logistic_class_weight`, que reduce falsos negativos aunque aumente falsas alarmas.

El cuarto problema es la explicabilidad. Aunque las variables están anonimizadas y no permiten una interpretación física directa, la explicabilidad sigue siendo útil. La importancia por permutación permite identificar qué señales influyen más en el rendimiento del modelo, detectar dependencias excesivas de variables concretas y orientar una revisión posterior con expertos de mantenimiento. Por tanto, la explicación no se interpreta como causalidad, sino como una auditoría del comportamiento predictivo del modelo.

El quinto problema es la robustez. En un sistema industrial, las señales pueden variar por ruido, cambios de sensores o condiciones de registro. La prueba con ruido no es una evaluación final nueva, sino una auditoría de sensibilidad que permite observar si el modelo se degrada ante perturbaciones de entrada.

El sexto problema es el shift o posible drift. El dataset no proporciona una validación temporal de producción, pero comparar train y test ayuda a revisar si las distribuciones son parecidas. En un despliegue real, esta dimensión sería crítica porque los camiones, sensores y patrones de mantenimiento pueden cambiar con el tiempo.

El séptimo problema son las anomalías. Los fallos APS pueden entenderse parcialmente como casos raros respecto al funcionamiento normal. Por eso se estudian detectores de anomalías como análisis auxiliar. Este enfoque no sustituye al modelo supervisado, pero puede aportar señales de alerta complementarias.

Fairness y privacidad no son el foco principal del proyecto porque las variables están anonimizadas y no hay atributos sensibles interpretables como sexo, edad, raza o grupo social. Esto impide analizar equidad por grupos de forma responsable. Aun así, la anonimización también limita la interpretación física. En un contexto interno de Scania, conocer el significado real de las variables permitiría validar mejor las explicaciones del modelo con expertos.

3.5. Flujo reproducible de notebooks

El proyecto está organizado como un flujo de notebooks numerados. El notebook `00_descarga_y_preparacion.ipynb` descarga o carga el dataset, convierte los valores `na`, codifica la variable objetivo y guarda las particiones procesadas. Este paso fija la base común para el resto del trabajo.

El notebook `01_EDA.ipynb` identifica las principales dificultades del dataset. En él se analiza la distribución de clases, los valores faltantes, las diferencias de missing por clase, las escalas, los outliers y las correlaciones. Estas observaciones justifican las decisiones metodológicas posteriores.

El notebook `02_modelado_supervisado.ipynb` contiene el modelado principal. Compara el baseline y los modelos supervisados mediante `GridSearchCV`, 5-fold CV y `Pipeline`. La selección del modelo principal se realiza solo con train.

El notebook `03_desbalanceo_y_coste.ipynb` estudia estrategias sensibles al desbalanceo y al coste. Su función no es cambiar el modelo final mirando test, sino analizar el intercambio entre PR-AUC, recall, falsos positivos, falsos negativos y coste.

El notebook `04_anomalias.ipynb` analiza detectores de anomalías entrenados sobre casos normales de train. Este análisis queda separado del modelo supervisado y se interpreta como una alerta secundaria.

El notebook `05_xai_robustez_errores.ipynb` audita el modelo principal con importancia por permutación, análisis de errores, robustez ante ruido y shift entre train y test. No modifica hiperparámetros, umbrales ni la selección del modelo.

4. Metodología

4.1. Conceptos técnicos usados

Antes de describir el protocolo experimental conviene aclarar varios conceptos que aparecen a lo largo de la memoria. Un **Pipeline** es una cadena de pasos que aplica siempre el mismo preprocesamiento y el mismo modelo. En este proyecto permite que la imputación de valores faltantes, el escalado cuando procede y el clasificador se entrenen como una unidad. Esto es importante porque evita que información de validación o test se use accidentalmente durante el ajuste.

GridSearchCV es una búsqueda sistemática de hiperparámetros. Para cada combinación propuesta, el método entrena y valida el modelo varias veces usando validación cruzada. La validación cruzada de 5 folds divide train en cinco partes. En cada repetición, cuatro partes se usan para entrenar y una para validar. El resultado final es una media de rendimiento más estable que una única partición interna.

El término *refit* indica que, una vez elegida la mejor configuración con validación cruzada, el estimador se vuelve a entrenar usando todo train. Así se aprovechan todos los datos de entrenamiento antes de evaluar en test. En este proyecto, test solo se usa después de este paso.

El baseline es un modelo de referencia simple. Sirve para comprobar si los modelos más complejos aportan valor real. Si un modelo avanzado no mejora claramente al baseline, no habría evidencia sólida de aprendizaje útil.

HistGradientBoosting es un modelo de boosting basado en árboles. Construye varios árboles de forma secuencial, de modo que cada nuevo árbol intenta corregir errores de los anteriores. Suele funcionar bien en datos tabulares porque puede capturar relaciones no lineales e interacciones entre variables.

La importancia por permutación es una técnica de explicabilidad global. Consiste en desordenar una variable y medir cuánto empeora la métrica del modelo. Si al desordenar una variable cae mucho la PR-AUC, esa variable es importante para el comportamiento predictivo. Esta técnica no demuestra causalidad. Solo indica dependencia estadística del modelo.

El análisis de anomalías busca observaciones raras respecto a un patrón normal. En este trabajo se entrena con casos negativos de train para comprobar si los fallos APS aparecen como casos anómalos. Es un análisis auxiliar, porque el objetivo principal sigue siendo la clasificación supervisada.

La robustez mide la sensibilidad del modelo ante cambios en la entrada. Aquí se añade ruido a variables numéricas para observar si el rendimiento se degrada. Esta prueba no sustituye a test ni cambia el modelo. Su función es auditar si el sistema depende de señales muy estables.

El shift o posible drift se refiere a cambios en la distribución de los datos. Shift describe diferencias entre particiones, como train y test. Drift suele usarse cuando ese cambio ocurre con el tiempo en producción. El dataset no permite afirmar deriva temporal estricta, pero sí permite comparar si train y test tienen distribuciones parecidas.

4.2. Protocolo experimental

El protocolo experimental es una parte central del proyecto. La partición oficial de train se usa para explorar, entrenar y seleccionar modelos. La partición oficial de test se reserva para la evaluación final. El test no se usa para ajustar hiperparámetros, no ajusta umbrales, no elige modelo y no elige estrategia.

Los modelos se comparan con **GridSearchCV** y validación cruzada estratificada de 5 folds sobre train. El preprocesamiento forma parte de **Pipeline**, por lo que la imputación y el escalado se ajustan dentro de cada fold. Esto evita fuga de información desde validación hacia entrenamiento.

Después de la búsqueda, **GridSearchCV** reentrena el mejor estimador con todo train mediante **refit**. Solo entonces se evalúa en test. Las tablas de test se interpretan como comparación final o como auditoría posterior, nunca como criterio de selección.

4.3. Modelos comparados

La Tabla 3 resume los modelos supervisados comparados.

Tabla 3: Modelos supervisados comparados

Familia	Modelos
Baseline	DummyClassifier
Modelos lineales	Regresión logística y SVM lineal
Árbol simple	Árbol de decisión
Ensembles	Random Forest, Extra Trees, AdaBoost e HistGradientBoosting

Todos los modelos usan imputación con mediana. La regresión logística y la SVM lineal incorporan además **StandardScaler**. La métrica principal de selección es **average_precision**, equivalente a PR-AUC. Como métricas complementarias se reportan recall positivo, F1 positivo, balanced accuracy, ROC-AUC y coste.

4.4. Métricas de evaluación

Las métricas se eligen teniendo en cuenta el desbalanceo y el coste operativo de los errores. La *accuracy* mide la proporción total de aciertos, pero puede ser engañosa porque la clase negativa domina el dataset. Por eso se usa solo como referencia general y no como criterio principal.

La *precisión positiva* mide qué proporción de las predicciones positivas son realmente fallos APS. Es útil para estimar cuántas alertas generadas por el sistema serían correctas. El *recall positivo* mide qué proporción de fallos APS reales se detectan. En este proyecto es especialmente importante porque un falso negativo implica dejar pasar un fallo real.

El *F1 positivo* combina precisión y recall de la clase positiva mediante su media armónica. Esta métrica resume el equilibrio entre generar alertas correctas y detectar suficientes fallos APS. La *balanced accuracy* promedia el rendimiento por clase y es más informativa que *accuracy* cuando hay desbalanceo.

La *ROC-AUC* mide la capacidad de ranking del modelo considerando la relación entre verdaderos positivos y falsos positivos a distintos umbrales. La *PR-AUC*, implementada como **average_precision**, resume la curva precisión-recall. Es la métrica principal de selección porque se centra en la clase positiva minoritaria y es más adecuada cuando los positivos son escasos.

El coste se calcula con la expresión de la Ecuación 1.

$$C = 10 \cdot FP + 500 \cdot FN \quad (1)$$

4.5. Reproducibilidad

La reproducibilidad se apoya en tres decisiones. La primera es la separación de responsabilidades entre notebooks. Cada etapa produce artefactos que la siguiente puede cargar, lo que evita depender de resultados manuales. La segunda es el uso de funciones comunes en **project_utils.py**. Este módulo define la carga del dataset, la conversión de etiquetas, el cálculo de métricas, la matriz de confusión y el coste. La tercera es la fijación de **RANDOM_STATE = 42** en los experimentos.

El entorno se documenta en **requirements.txt**. Las dependencias principales son **numpy**, **pandas**, **scikit-learn**, **matplotlib**, **seaborn**, **joblib**, **ucimlrepo**, **notebook** y **nbconvert**. El repositorio incluye un **README.md** con instrucciones de instalación y orden de ejecución. Esto permite reconstruir el flujo desde una máquina limpia, siempre que los datos o su descarga estén disponibles.

Desde el punto de vista experimental, la reproducibilidad también exige no usar test durante la toma de decisiones. Por ello, las tablas de test se generan una vez fijados modelos, hiperparámetros y estrategias. Cualquier ajuste futuro de coste o umbral debería hacerse con validación interna en train, no con test.

5. Experimentos y resultados

5.1. Modelado supervisado

5.1.1. Resultados de validación cruzada

La Tabla 4 muestra los resultados medios de validación cruzada sobre train. La tabla está ordenada por PR-AUC, que es la métrica principal de selección.

Tabla 4: Comparativa de modelos con 5-fold CV sobre train

Modelo	PR-AUC	Recall pos.	F1 pos.	Bal. acc.	Tiempo s
HistGradientBoosting	0.881	0.738	0.809	0.868	35.9
Random Forest	0.874	0.697	0.787	0.848	145.5
Extra Trees	0.865	0.702	0.793	0.850	34.2
AdaBoost	0.794	0.665	0.730	0.831	172.3
Logistic Regression	0.772	0.632	0.708	0.815	31.3
Linear SVM	0.769	0.619	0.708	0.808	1328.0
Decision Tree	0.706	0.639	0.698	0.818	50.0
Dummy	0.017	0.021	0.021	0.502	5.0

El baseline confirma que una estrategia simple casi no detecta positivos. El modelo **Dummy** obtiene una PR-AUC de 0.017 y un recall positivo de 0.021, por lo que sirve para mostrar que la accuracy no sería una métrica suficiente en este problema. Frente a esta referencia, todos los modelos entrenados aprenden señal útil del dataset.

HistGradientBoosting obtiene el mejor resultado global, con una PR-AUC media de 0.881 y el mejor F1 positivo, con 0.809. Esto lo convierte en el modelo seleccionado según la métrica principal. Sin embargo, la diferencia con otros ensembles no es extrema. **Random Forest** alcanza una PR-AUC de 0.874 y queda como segundo mejor modelo, aunque tarda 145.5 segundos, bastante más que **HistGradientBoosting**. **Extra Trees** también es una alternativa competitiva, con PR-AUC de 0.865, F1 positivo de 0.793 y un tiempo de 34.2 segundos, muy similar al modelo seleccionado.

Los modelos lineales ofrecen una referencia útil, pero quedan por debajo de los ensembles. **Logistic Regression** obtiene una PR-AUC de 0.772 con un tiempo bajo, lo que la convierte en un modelo simple y razonable como comparación. En cambio, **Linear SVM** no resulta eficiente en este experimento, ya que tarda alrededor de 1328 segundos y no mejora a la regresión logística ni a los ensembles.

En conjunto, la comparación muestra que **HistGradientBoosting** ofrece el mejor equilibrio entre rendimiento y coste computacional. **Random Forest** es muy competitivo, pero más lento. **Extra Trees** destaca como alternativa rápida y sólida. Los modelos lineales y el árbol simple son útiles como referencia, pero no alcanzan el rendimiento de los ensembles principales.

5.1.2. Selección del modelo principal

El modelo principal es `hist_gradient_boosting`. La selección se hace solo con validación cruzada en train y usando PR-AUC. El valor de selección guardado en los artefactos es 0.881018. Esta decisión se toma antes de analizar el rendimiento final en test.

5.2. Resultados finales en test

Una vez seleccionado el modelo principal, se evalúa en test. La Tabla 5 muestra la comparativa final de los modelos ya ajustados.

Tabla 5: Comparativa final en test

Modelo	PR-AUC	Recall pos.	F1 pos.	FP	FN	Coste
HistGradientBoosting	0.922	0.752	0.842	13	93	46630
Random Forest	0.920	0.717	0.816	15	106	53150
Extra Trees	0.918	0.704	0.805	17	111	55670
AdaBoost	0.840	0.680	0.753	47	120	60470
Logistic Regression	0.819	0.701	0.767	48	112	56480
Linear SVM	0.822	0.645	0.746	32	133	66820
Decision Tree	0.770	0.645	0.748	30	133	66800
Dummy	0.024	0.016	0.018	272	369	187220

La Tabla 5 confirma que el comportamiento observado en validación cruzada se mantiene en test. **HistGradientBoosting** obtiene el mejor resultado global, con PR-AUC de 0.922, F1 positivo de 0.842 y recall positivo de 0.752. Por eso se mantiene como modelo principal.

Otros ensembles, como **Random Forest** y **Extra Trees**, no quedan muy lejos y también muestran un rendimiento sólido. Sin embargo, **HistGradientBoosting** combina mejor PR-AUC, F1 positivo, menor número de falsos positivos y menor coste entre los modelos supervisados principales.

El baseline vuelve a quedar muy por debajo, lo que confirma que el problema requiere modelos capaces de aprender patrones reales en las variables. La matriz de confusión del modelo seleccionado se muestra en la Figura 6.

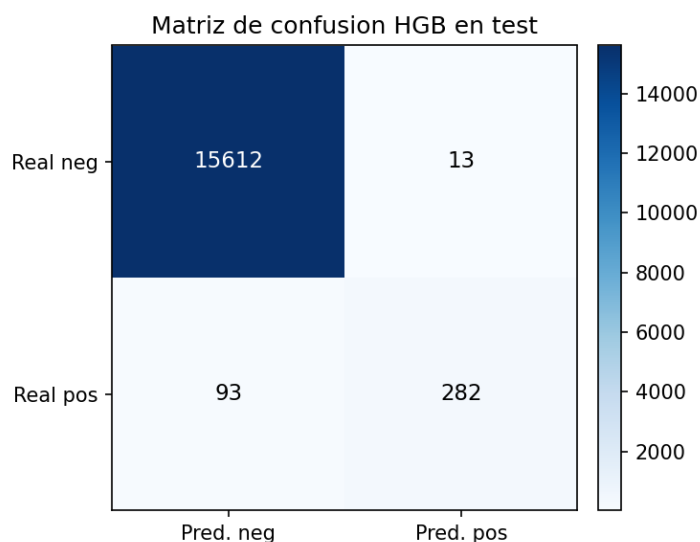


Figura 6: Matriz de confusión del modelo **HistGradientBoosting** en test

La Figura 6 resume visualmente los aciertos y errores del modelo final. La diagonal principal representa las predicciones correctas. En ella aparecen los verdaderos negativos y los verdaderos positivos. El modelo clasifica correctamente 15612 casos negativos y detecta 282 fallos APS.

Los errores aparecen fuera de esa diagonal. Hay 13 falsos positivos, que son camiones marcados como fallo APS aunque no lo eran, y 93 falsos negativos, que son fallos APS reales que el modelo no detecta. Estos últimos son el punto más delicado porque tienen mayor impacto operativo.

5.3. Desbalanceo y coste

El notebook de desbalanceo y coste no sustituye al modelo principal. Su función es estudiar cómo cambian las métricas cuando se introducen estrategias sensibles al desbalanceo. Las

estrategias comparadas son `hgb_sin_ajuste`, `logistic_sin_ajuste`, `logistic_class_weight` y `rf_class_weight`.

La selección por PR-AUC en validación cruzada vuelve a elegir `hgb_sin_ajuste`, con PR-AUC media de 0.881. La Tabla 6 muestra la comparación final en test de estas estrategias.

Tabla 6: Comparativa de estrategias de desbalanceo y coste en test

Estrategia	PR-AUC	Recall pos.	F1 pos.	FP	FN	Coste
<code>hgb_sin_ajuste</code>	0.922	0.752	0.842	13	93	46630
<code>rf_class_weight</code>	0.880	0.573	0.712	14	160	80140
<code>logistic_sin_ajuste</code>	0.819	0.701	0.767	48	112	56480
<code>logistic_class_weight</code>	0.800	0.925	0.639	364	28	17640

`logistic_class_weight` reduce los falsos negativos de 93 a 28. A cambio aumenta los falsos positivos de 13 a 364 y reduce F1 positivo. Como el coste penaliza mucho más los falsos negativos, su coste total baja a 17640 frente a 46630 de `HistGradientBoosting`.

Este resultado no cambia el modelo principal. La razón es metodológica. El modelo principal se seleccionó por PR-AUC en train antes de mirar test. Si el objetivo principal fuese minimizar coste, habría que diseñar un experimento donde el coste se optimizara dentro de train mediante validación cruzada. No sería correcto escoger `logistic_class_weight` como modelo final solo porque su coste en test sea menor.

5.4. Análisis de anomalías

El análisis de anomalías es secundario y no sustituye al modelo supervisado. Su objetivo es comprobar si los fallos APS aparecen como observaciones raras frente a los casos normales. En este contexto, un detector de anomalías es un modelo que aprende cómo son los casos normales y asigna una puntuación de rareza a cada nueva observación.

Los detectores se entrenan con casos normales de train. Después, los casos que se alejan más de ese comportamiento normal se marcan como posibles anomalías. Los umbrales se calculan usando train y test se utiliza solo para evaluar. Se comparan Isolation Forest, Local Outlier Factor y One-Class SVM. La Tabla 7 resume los resultados.

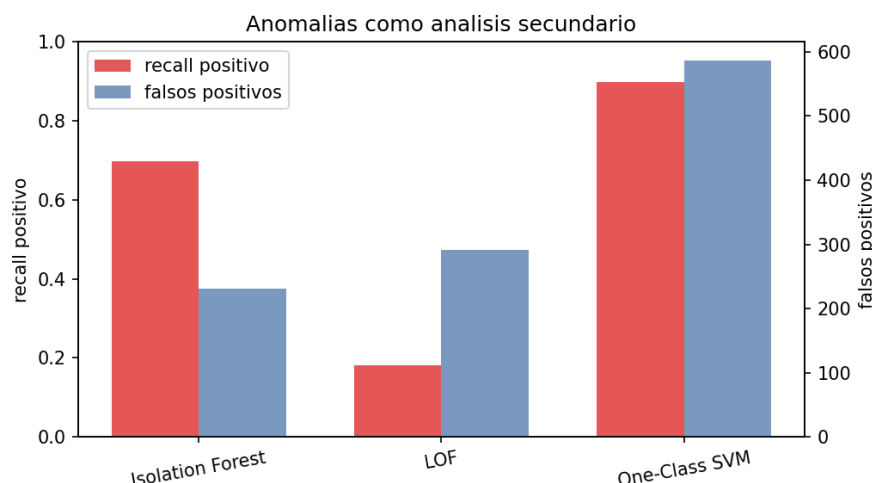


Figura 7: Comparación de recall positivo y falsos positivos en detectores de anomalías

La Figura 7 muestra el intercambio principal de los detectores de anomalías. One-Class SVM alcanza el mayor recall positivo, pero también genera muchos más falsos positivos. Respecto a la comparativa global, la Tabla 7 resume los resultados de los detectores en test.

Tabla 7: Resultados de detección de anomalías en test

Detector	PR-AUC	Recall pos.	F1 pos.	FP	FN	Coste
Isolation Forest	0.568	0.699	0.604	231	113	58810
Local Outlier Factor	0.146	0.181	0.185	291	307	156410
One-Class SVM	0.589	0.899	0.519	586	38	24860

La Tabla 7 no representa una nueva selección del modelo principal, sino una comparación secundaria con otro enfoque. A diferencia del clasificador supervisado, estos detectores no aprenden directamente a distinguir entre casos APS y no APS. Se entrenan con casos normales para aprender el comportamiento habitual del camión y después marcan como sospechosos los casos que se alejan de ese patrón.

Este análisis sirve para comprobar si un fallo APS también puede verse como un comportamiento raro del camión. Por eso los detectores de anomalías podrían usarse como una alerta auxiliar separada del modelo principal, pero no como sustituto directo.

En esta comparación, **One-Class SVM** detecta muchos positivos y reduce los falsos negativos a 38. Sin embargo, genera 586 falsos positivos. Esto significa que deja escapar menos fallos, pero también enviaría más camiones a revisión. Por tanto, puede ser útil si se quiere priorizar la detección, aunque implique aceptar más falsas alarmas.

5.5. Explicabilidad, robustez y errores

5.5.1. Importancia por permutación

La importancia por permutación se calcula sobre una muestra de train y mide cuánto cae average precision cuando se desordena una variable. Si al alterar una variable el rendimiento baja mucho, significa que el modelo dependía bastante de esa información para predecir. La Tabla 8 muestra las variables más influyentes.

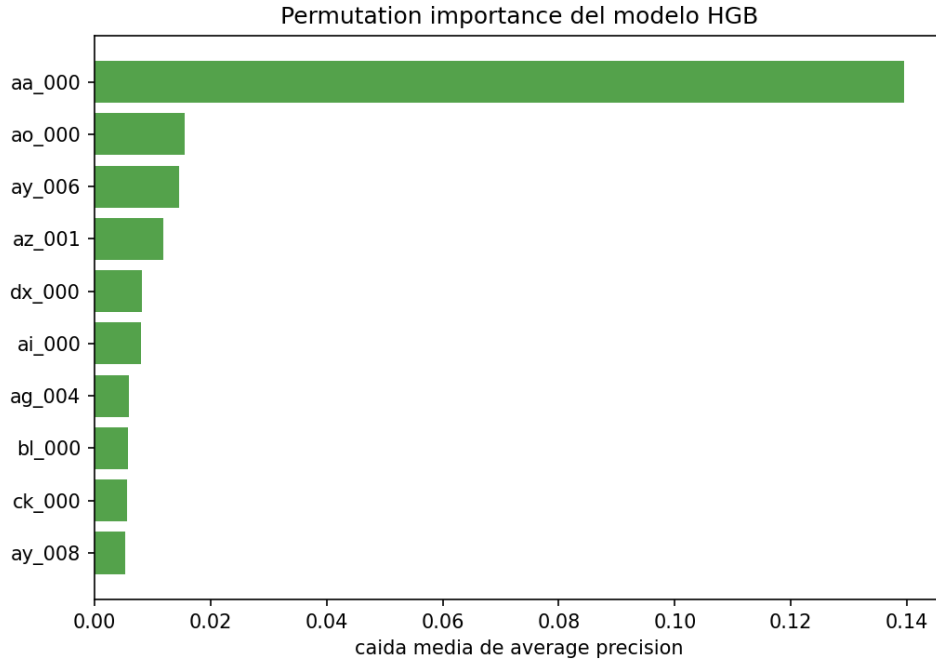


Figura 8: Importancia por permutación del modelo principal

La Figura 8 permite ver que la importancia está muy concentrada. **aa_000** destaca con mucha diferencia sobre el resto, lo que conviene tener en cuenta al interpretar la estabilidad del modelo.

Tabla 8: Variables más importantes por permutation importance

Variable	Importancia media	Desv.
aa_000	0.1394	0.0252
ao_000	0.0156	0.0013
ay_006	0.0145	0.0080
az_001	0.0118	0.0043
dx_000	0.0082	0.0041
ai_000	0.0080	0.0010
ag_004	0.0059	0.0063
bl_000	0.0059	0.0051
ck_000	0.0056	0.0006
ay_008	0.0054	0.0038

aa_000 domina claramente el ranking. Al permutar esta variable, la average precision cae alrededor de 0.139, lo que indica que el modelo pierde bastante capacidad predictiva cuando esa información se desordena. La segunda variable, **ao_000**, queda muy por debajo con una caída de 0.016, por lo que la importancia está muy concentrada en **aa_000**.

Este resultado sugiere que el modelo depende mucho de una señal principal. Debido a la anonimización, no se puede interpretar **aa_000** como una causa física del fallo APS. Aun así, la importancia por permutación sí permite auditar el comportamiento del modelo e identificar qué señales deberían revisarse con mayor atención si se conociera su significado real.

5.5.2. Análisis de errores

El análisis de errores del modelo final muestra 15894 aciertos, 93 falsos negativos y 13 falsos positivos sobre 16000 observaciones de test. Un ejemplo local revisado en el notebook corresponde a un falso negativo. El caso real es positivo, pero el modelo lo predice como negativo. En ese ejemplo se inspeccionan los valores de las diez variables más importantes para entender el error desde las señales globales del modelo.

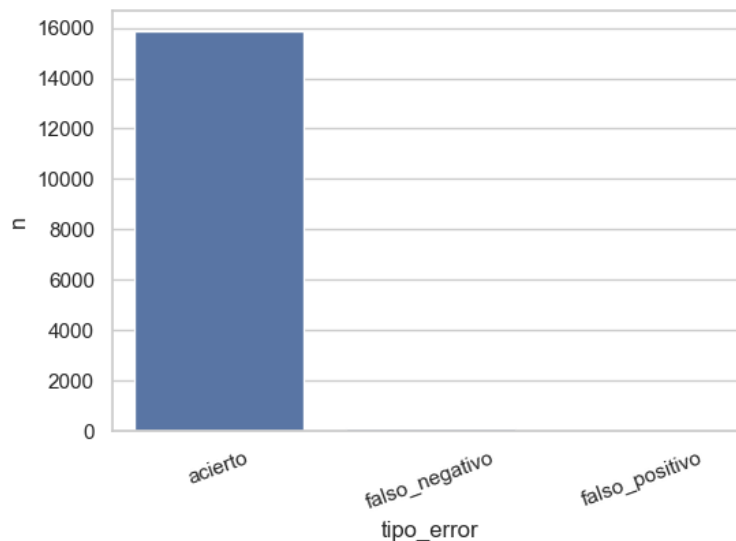


Figura 9: Resumen de aciertos, falsos negativos y falsos positivos en test

La Figura 9 resume la distribución general de aciertos y errores en test. Aunque los errores son pocos en relación con el tamaño total de la partición, la matriz de confusión permite concretar su impacto. El modelo produce 13 falsos positivos y 93 falsos negativos.

La principal limitación operativa sigue siendo el falso negativo. Cada uno de esos 93 casos representa un fallo APS real que el modelo no detecta. Esta diferencia es relevante porque, según la función de coste definida, un falso negativo pesa mucho más que un falso positivo.

5.5.3. Robustez ante ruido

La prueba de robustez añade ruido a variables numéricas en una muestra de train. Es una auditoría de sensibilidad posterior al modelado. La Tabla 9 resume la degradación observada.

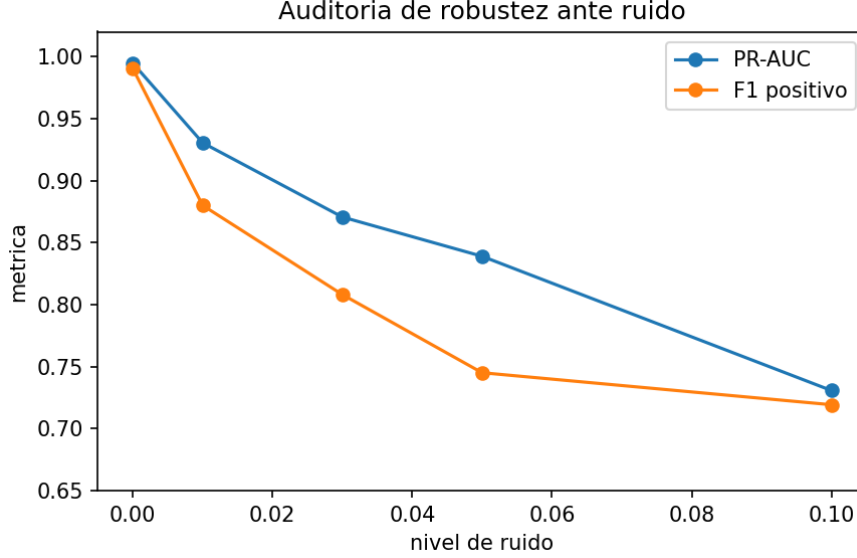


Figura 10: Auditoría de sensibilidad ante ruido en variables numéricas

La Figura 10 muestra una caída progresiva de PR-AUC y F1 positivo al aumentar el nivel de ruido. Con ruido bajo el modelo mantiene un rendimiento razonable, pero a medida que las variables se perturban más, ambas métricas descienden de forma clara. Esto indica que el modelo depende de la calidad de las señales de entrada. En un uso real sería importante monitorizar sensores, valores anómalos y cambios en la captura de datos para evitar una degradación del rendimiento.

Tabla 9: Robustez ante ruido en variables numéricas

Ruido	PR-AUC	Recall pos.	F1 pos.	FP	FN
0.00	0.995	0.981	0.990	0	1
0.01	0.930	0.846	0.880	4	8
0.03	0.871	0.769	0.808	7	12
0.05	0.839	0.731	0.745	12	14
0.10	0.731	0.788	0.719	21	11

La tabla complementa la figura anterior mostrando los mismos resultados con valores concretos. Se observa que al aumentar el ruido la PR-AUC baja de 0.995 a 0.731 y el F1 positivo baja de 0.990 a 0.719. Además, los falsos positivos aumentan de 0 a 21. Esto refuerza que el modelo es sensible a la calidad de las señales de entrada.

5.5.4. Shift entre train y test

El shift entre train y test se analiza con el estadístico KS por variable. Este estadístico compara si una misma variable tiene una distribución parecida en train y en test. En otras palabras,

comprueba si los valores de una columna se reparten de forma similar en ambas particiones. Un valor KS cercano a 0 indica que las distribuciones son muy parecidas. Un valor más alto indicaría que esa variable se comporta de forma diferente en train y en test.

Tabla 10: Variables con mayor diferencia de distribución entre train y test según KS

Variable	KS	p-value
br_000	0.0227	0.1906
bq_000	0.0208	0.2515
ab_000	0.0152	0.5152
bp_000	0.0145	0.6432
cz_000	0.0124	0.1024
cb_000	0.0120	0.0555
ay_008	0.0118	0.0599
bj_000	0.0118	0.0597

La Tabla 10 muestra las variables donde train y test se diferencian más según este estadístico. Aun así, los valores son bajos. La mayor diferencia aparece en `br_000`, con KS de 0.0227, seguida de `bq_000`, con 0.0208. Esto significa que incluso las variables más distintas tienen distribuciones bastante parecidas entre train y test.

El p-value ayuda a interpretar si la diferencia observada es estadísticamente clara. En las primeras variables, los p-values no son especialmente bajos, por lo que no hay evidencia fuerte de que train y test estén separados de forma importante en esas columnas.

La interpretación debe ser prudente. Este análisis no demuestra que no pueda haber deriva en producción, porque el dataset no representa un flujo temporal real. Solo permite comprobar que las particiones usadas en el proyecto parecen razonablemente parecidas desde el punto de vista distributivo.

6. Discusión

6.1. Por qué HistGradientBoosting es el modelo principal

`HistGradientBoosting` es el modelo principal porque fue seleccionado por PR-AUC media en validación cruzada usando solo train. Además, mantiene el mejor rendimiento global en test entre los modelos comparados. Supera claramente al baseline, obtiene buen F1 positivo y genera muy pocos falsos positivos.

El resultado también es razonable desde el punto de vista técnico. Los modelos de boosting suelen funcionar bien en datos tabulares con relaciones no lineales e interacciones entre variables. En este proyecto, además, ofrece un buen equilibrio entre rendimiento y tiempo de entrenamiento.

6.2. PR-AUC frente a coste

PR-AUC y coste responden a preguntas distintas. PR-AUC evalúa la capacidad del modelo para ordenar positivos en un problema desbalanceado. El coste penaliza errores concretos con pesos distintos. Por eso `HistGradientBoosting` puede ser mejor por PR-AUC y `logistic_class_weight` puede ser mejor por coste en la comparativa final.

La conclusión correcta no es cambiar el modelo final mirando test. La lectura adecuada es que existe una alternativa operativa si el objetivo principal cambia. Para defender `logistic_class_weight` como modelo final habría que seleccionarlo dentro de train con una métrica de coste o con un protocolo de umbrales validado sin test.

Esta diferencia es importante para la toma de decisiones. Si el sistema se usa para priorizar revisiones con capacidad limitada, puede interesar un modelo con mayor precisión y pocas falsas alarmas. En ese caso, `HistGradientBoosting` ofrece un equilibrio razonable. Si el coste de

dejar pasar fallos es muy alto y existe capacidad para revisar muchas alertas, una estrategia como `logistic_class_weight` puede ser más atractiva. La memoria separa ambas lecturas porque responden a objetivos operativos distintos.

6.3. Explicabilidad con variables anonimizadas

La explicabilidad tiene un papel particular en este dataset. Las variables no describen directamente componentes interpretables para una persona externa al fabricante. Por tanto, no se puede afirmar que una variable concreta sea la causa física del fallo. Aun así, la importancia por permutación aporta una lectura útil del comportamiento predictivo.

El resultado más relevante es la dependencia de `aa_000`. Esta variable domina el ranking de importancia, lo que puede tener dos lecturas. Por un lado, puede ser una señal muy informativa para detectar fallos APS. Por otro lado, una dependencia excesiva de una sola señal puede aumentar la fragilidad si esa variable cambia, se registra mal o tiene un comportamiento distinto en producción.

La explicabilidad también ayuda a orientar una revisión posterior con expertos de mantenimiento. Un experto con acceso al significado real de las variables podría comprobar si las señales destacadas son coherentes con el funcionamiento del sistema APS. Además, el análisis de falsos positivos y falsos negativos permite revisar casos donde el modelo se equivoca y estudiar si el error se relaciona con patrones de variables importantes, missing values o condiciones anómalas.

6.4. Limitaciones

La primera limitación es la anonimización de variables. Se pueden detectar patrones estadísticos, pero no se puede explicar físicamente cada señal. La segunda limitación es el fuerte desbalanceo, que hace que pequeñas variaciones en falsos negativos tengan impacto relevante. La tercera limitación es el coste simplificado. La penalización de 10 y 500 es útil para comparar, pero no representa todos los costes reales de taller, logística y seguridad.

También hay limitaciones de evaluación. El proyecto usa una partición oficial de test, pero no valida el sistema en producción. No hay monitorización continua, validación temporal ni revisión por expertos de mantenimiento. Las pruebas de anomalías, robustez y shift son auditorías complementarias, no sustitutos de un despliegue real.

Fairness y privacidad se tratan como limitaciones de alcance. No se han identificado atributos sensibles interpretables, por lo que no se puede medir discriminación por grupos sociales. Tampoco se trabaja con datos personales directos en la memoria, ya que las variables están anonimizadas. Aun así, un despliegue real en una empresa debería revisar políticas de acceso, trazabilidad de alertas y gobierno de datos.

6.5. Trabajo futuro

Una mejora natural sería optimizar coste dentro de validación cruzada en train. Otra mejora sería estudiar umbrales de decisión con un conjunto de validación interno o validación cruzada, siempre sin usar test. También sería útil realizar validación temporal si se dispone de fechas, calibrar probabilidades y añadir revisión experta de falsos negativos.

En producción, sería necesario monitorizar drift, missing values, estabilidad de sensores, tasas de falsos negativos y coste real de intervención. También habría que diseñar una integración con procesos de mantenimiento y definir quién revisa las alertas.

7. Conclusiones

El proyecto construye un flujo completo para detectar fallos APS en camiones Scania a partir de datos tabulares industriales. El dataset presenta dificultades reales como clase positiva minoritaria, valores faltantes, escalas diferentes, outliers, variables correlacionadas y anonimización.

Los modelos se comparan con `GridSearchCV`, 5-fold CV y `Pipeline`. La selección de hiperparámetros se hace solo con train. El mejor estimador se reentrena con todo train mediante `refit`. Test se usa solo para evaluación final y auditoría posterior. No elige modelo, estrategia, umbral ni hiperparámetros.

El modelo principal es `HistGradientBoosting`. Se selecciona por PR-AUC en train, obtiene PR-AUC cercana a 0.881 en validación cruzada y mantiene PR-AUC cercana a 0.922 en test. En la evaluación final alcanza F1 positivo cercano a 0.842 y recall positivo cercano a 0.752. La matriz de confusión muestra 15612 verdaderos negativos, 282 verdaderos positivos, 13 falsos positivos y 93 falsos negativos.

El análisis de coste muestra que los falsos negativos siguen siendo importantes. `logistic_class_weight` reduce los falsos negativos a 28 y baja el coste total a 17640, pero aumenta mucho los falsos positivos y no fue el modelo seleccionado por la métrica principal en train. Por tanto, queda como alternativa operativa y no como modelo final.

Los análisis de anomalías, explicabilidad, robustez, errores y shift aportan una visión más completa. Las anomalías sugieren que algunos fallos APS pueden comportarse como observaciones raras y podrían estudiarse como sistema auxiliar de alerta. La importancia por permutación muestra una dependencia fuerte de `aa_000`. La robustez ante ruido evidencia sensibilidad a perturbaciones. El análisis de errores confirma que el modelo es útil, pero no perfecto.

En conjunto, el trabajo ofrece una solución técnica defendible para un problema real de mantenimiento predictivo. Su principal fortaleza no es solo el resultado de test, sino la separación estricta entre selección y evaluación, la comparación de modelos, la discusión de coste y la auditoría posterior del comportamiento.

Tabla 11: Resumen final de conclusiones metodológicas

Aspecto	Conclusión
Modelo principal	<code>HistGradientBoosting</code>
Métrica de selección	PR-AUC media en 5-fold CV sobre train
Criterio metodológico	Selección con <code>GridSearchCV</code> y 5-fold CV en train. Test reservado solo para evaluación final
Mejor alternativa por coste	<code>logistic_class_weight</code> , con 28 falsos negativos y coste 17640 en test
Principal limitación	Variables anonimizadas, clase positiva minoritaria y coste simplificado
Trabajo futuro	Optimizar coste y umbrales dentro de train, validación temporal, calibración y revisión experta

8. Referencias

- [1] Dua, D. y Graff, C. APS Failure at Scania Trucks. UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences. <https://archive.ics.uci.edu/dataset/421/aps+failure+at+scania+trucks>
- [2] Lozano Ortega, M. A. Aprendizaje Avanzado. Apuntes de la asignatura Aprendizaje Avanzado del Grado en Ingeniería en Inteligencia Artificial. Departamento de Ciencia de la Computación e Inteligencia Artificial, Universidad de Alicante. Curso 2025–2026. Material disponible bajo licencia Creative Commons Attribution 4.0 International. <https://malozano.github.io/aprendizaje-avanzado/>

A. Anexo A. Archivos principales del flujo

Tabla 12: Archivos principales del flujo experimental

Archivo	Uso en la memoria
README.md	Resumen del proyecto, protocolo, resultados principales e instrucciones de uso.
memoria_scania_aps.tex	Fuente en LaTeX de la memoria del proyecto.
project_utils.py	Constantes de coste, funciones de carga, métricas, matriz de confusión y cálculo de coste.
requirements.txt	Dependencias necesarias para reproducir el entorno de ejecución.
data/	Carpeta de datos originales y procesados utilizados por los notebooks.
artifacts/	Carpeta con métricas, tablas, figuras y modelos generados durante la ejecución.
00_descarga_y_preparacion.ipynb	Carga, limpieza inicial, conversión de valores <code>na</code> , codificación de clases y guardado de particiones procesadas.
01_EDA.ipynb	Análisis exploratorio de desbalanceo, missing values, outliers, correlaciones y métricas adecuadas.
02_modelado_supervisado.ipynb	Comparación de modelos con Pipeline, GridSearchCV, CV, selección de HGB y evaluación final en test.
03_desbalanceo_y_coste.ipynb	Estudio de estrategias de desbalanceo, coste asimétrico y comparación con <code>logistic_class_weight</code> .
04_anomalias.ipynb	Detectores de anomalías entrenados sobre casos normales y evaluación secundaria como alerta auxiliar.
05_xai_robustez_errores.ipynb	Importancia por permutación, análisis de errores, robustez ante ruido y shift entre train y test.

B. Anexo B. Reproducción

Para reproducir el proyecto se recomienda crear un entorno limpio e instalar las dependencias.

```
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
```

En Linux o macOS, la activación cambia a `source .venv/bin/activate`. También puede usarse un entorno conda equivalente instalando `requirements.txt`.

Después se ejecutan los notebooks desde la raíz del proyecto.

```
jupyter nbconvert --to notebook --execute 00_descarga_y_preparacion.ipynb --inplace
jupyter nbconvert --to notebook --execute 01_EDA.ipynb --inplace
jupyter nbconvert --to notebook --execute 02_modelado_supervisado.ipynb --inplace
jupyter nbconvert --to notebook --execute 03_desbalanceo_y_coste.ipynb --inplace
jupyter nbconvert --to notebook --execute 04_anomalias.ipynb --inplace
jupyter nbconvert --to notebook --execute 05_xai_robustez_errores.ipynb --inplace
```